

SOCIAL NETWORKS

ASSIGNMENT 2

Amina Ferra
2205225

Graph-based Bot Detection with Structural Evasion & Poisoning Attacks

Assignment Overview

The goal was to:

- ✓ Build a social network graph from the Facebook dataset.
- ✓ Compute essential graph metrics (degree, clustering coefficient, betweenness centrality, eigenvector centrality, communities).
- ✓ Create synthetic labels to identify bots vs humans.
- ✓ Train baseline bot detection models (GraphSAGE & GCN).
- ✓ Simulate Structural Evasion Attacks and Graph Poisoning Attacks to evaluate model robustness.
- ✓ Compare performance metrics before and after attacks.
- ✓ Visualize the graph and ego networks highlighting the impact of attacks.

Explanation of the Models

GraphSAGE

GraphSAGE (Graph Sample and Aggregate) is a GNN that learns node embeddings by sampling neighbors and aggregating their features. It's designed to handle large graphs and generalize to unseen nodes.

Steps:

- ✓ For each node, GraphSAGE grabs a fixed number of neighbors.
- ✓ Aggregates them (mean / max-pool).
- ✓ Combines neighbor info + node's own features.
- ✓ Learns a representation that captures structure + attributes.

GCN (Graph Convolutional Network)

GCN applies graph convolution, which mixes a node's features with all its neighbors at once using normalized adjacency.

Steps:

- ✓ Each node receives a weighted "message" from all of its neighbors.
- ✓ These messages are averaged using graph Laplacian normalization.
- ✓ Feeds through layers → final node classification.

Explanation of the Attacks

Structural Evasion Attack

Goal: Bots try to hide by changing their local structure to look more “human”.

How the attack works:

- ✓ Each bot connects to multiple human nodes.
- ✓ The bot’s degree increases and becomes like human degree.
- ✓ Clustering coefficient + centrality become more “normal”.

Effect on detection model:

- ✓ Bots blend into the social graph.
- ✓ Their structural patterns look less unusual.
- ✓ The model’s precision/recall for bots drops.

This attack modifies the graph used for testing, not training.

Graph Poisoning Attack

Goal: Manipulate the **training data** so the model learns the wrong patterns.

How the attack works:

- ✓ During training, bots gain many edges to humans.
- ✓ The model sees bots that *look human* during training.
- ✓ Later, on clean test data, it becomes confused.

Effect on your model:

- ✓ The poisoned training graph teaches wrong embeddings.
- ✓ Even clean test bots get misclassified.
- ✓ Bigger drop in performance than evasion.

This attack directly corrupts the model’s learning process, not just the test graph.

Code Explanation

a. Graph Loading & Preprocessing

```
# Data Loading and Graph Building
def load_facebook_graph(file_path="Data/facebook_combined.txt"):
    try:
        graph = nx.read_edgelist(file_path, nodetype=int)
        if graph.is_directed():
            graph = graph.to_undirected()
        if not nx.is_connected(graph):
            graph = graph.subgraph(max(nx.connected_components(graph), key=len)).copy()
        print(f"Graph loaded successfully (LCC used): {graph.number_of_nodes()} nodes, {graph.number_of_edges()} edges.")
        return graph
    except FileNotFoundError:
        print(f"Error: File not found at {file_path}. Returning an empty graph.")
        return nx.Graph()

# Calculate and Save Descriptive Stats
def calculate_and_save_stats(df_features, feature_cols, scenario_name):
    stats_df = df_features[feature_cols].agg(['mean', 'min', 'max']).T
    descriptive_stats = df_features.describe().T
    print(f"\nDescriptive Statistics for {scenario_name} Graph Metrics:")
    print(stats_df.round(6).to_markdown(numalign="left", stralign="left"))
    save_dataframe_to_csv(descriptive_stats.round(6), METRICS_FOLDER, f'03_{scenario_name.lower().replace(" ", "_")}_features_s
    return stats_df
```

- ✓ Load Facebook egonet dataset.
- ✓ Extract 4039 nodes, 88234 edges.
- ✓ Compute 5 metrics: degree, clustering coeff, betweenness, eigenvector centrality, and community IDs.
- ✓ Save computed features for further use.

b. Synthetic Labels Creation

```
def compute_graph_features(graph):
    clustering = nx.clustering(graph)
    k_nodes = min(graph.number_of_nodes(), 500)
    betweenness = nx.betweenness_centrality(graph, k=k_nodes, seed=42, normalized=True)
    try:
        eigenvector = nx.eigenvector_centrality(graph, max_iter=200, tol=1e-05)
    except nx.PowerIterationFailedConvergence:
        print("Warning: Eigenvector Centrality failed to converge. Using fallback uniform values.")
        eigenvector = {node: 1.0 / graph.number_of_nodes() for node in graph.nodes()}
    partition = community_louvain.best_partition(graph, random_state=42)
    features_df = pd.DataFrame({
        'degree': pd.Series(degree),
        'clustering_coeff': pd.Series(clustering),
        'betweenness_centrality': pd.Series(betweenness),
        'eigenvector_centrality': pd.Series(eigenvector),
        'community_id': pd.Series(partition),
    }).fillna(0)
    all_nodes = list(graph.nodes())
    features_df = features_df.reindex(all_nodes, fill_value=0)
    print(f"\nMetric Output Sample (Node IDs are indices):\n{features_df.head().round(6)}\n")
    return features_df

def create_synthetic_labels(graph, features_df, bot_percentage=0.05):
    N = graph.number_of_nodes()
    bot_count = max(1, int(bot_percentage * N))
    sorted_by_degree = features_df['degree'].sort_values()
    low_degree_bots = sorted_by_degree.head(bot_count // 2).index.tolist()
    high_degree_bots = sorted_by_degree.tail(bot_count - bot_count // 2).index.tolist()
    bot_nodes = set(low_degree_bots + high_degree_bots)
    labels = pd.Series(0, index=graph.nodes())
    labels.loc[list(bot_nodes)] = 1
    print(f"Synthetic labels created: {len(bot_nodes)} bots (1). Total nodes: {N}.")
    return features_df.join(labels.rename('label'), bot_nodes)
```

- ✓ Label nodes as bots (class 1) or humans (class 0) based on degree extremes (low-degree and high-degree nodes as bots).

- ✓ Resulted in 201 bots out of 4039 nodes.
- ✓ Resulted in 3,838 humans out of 4039 nodes.

c. Baseline Model Training

```
# GNN Model Definitions
class GCNModel(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
        super().__init__()
        self.conv1 = GCNConv(in_channels, hidden_channels)
        self.conv2 = GCNConv(hidden_channels, out_channels)
    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.5, training=self.training)
        x = self.conv2(x, edge_index)
        return F.log_softmax(x, dim=1)

class GraphSAGEModel(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
        super().__init__()
        self.conv1 = SAGEConv(in_channels, hidden_channels)
        self.conv2 = SAGEConv(hidden_channels, out_channels)
    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.5, training=self.training)
        x = self.conv2(x, edge_index)
        return F.log_softmax(x, dim=1)

# Training and Evaluation Functions
def train_gnn(model, data, epochs=100, model_name=""):
    train_labels = data.y[data.train_mask]
    bot_count = (train_labels == 1).sum().item()
    human_count = (train_labels == 0).sum().item()
    class_weight = human_count / bot_count if bot_count > 0 else 1.0
```

- ✓ Prepare features by normalizing metrics.
- ✓ Train two GNN models: GraphSAGE and GCN on clean data for 100 epochs with weighted loss for class imbalance.
- ✓ Evaluate on test set: accuracy, F1-score, recall for both bots and humans.

```

# Attack Implementation
def structural_evasion_attack_nx(original_graph, target_indices_pg, data_df_baseline, idx_to_node, test_human_indices_pg, max_d):
    evasion_graph = original_graph.copy()
    human_degree_mean = data_df_baseline[data_df_baseline['label'] == 0]['degree'].mean()
    target_nodes_nx = map_indices_to_nodes(target_indices_pg, idx_to_node)
    test_human_nodes_nx = map_indices_to_nodes(test_human_indices_pg, idx_to_node)
    total_links_added = 0
    for bot_node in target_nodes_nx:
        current_degree = evasion_graph.degree(bot_node)
        degree_diff = int(human_degree_mean - current_degree)
        if degree_diff > 0:
            potential_neighbors = [
                n for n in evasion_graph.nodes()
                if n in test_human_nodes_nx and not evasion_graph.has_edge(bot_node, n)
            ]
            num_changes = min(degree_diff, max_changes, len(potential_neighbors))
            if num_changes > 0:
                edges_to_add = random.sample(potential_neighbors, num_changes)
                evasion_graph.add_edges_from([(bot_node, n) for n in edges_to_add])
                total_links_added += num_changes
    print(f"Structural Evasion Attack: Added {total_links_added} links to {len(target_nodes_nx)} bots.")
    return evasion_graph

def graph_poisoning_attack_nx(original_graph, train_bot_indices_pg, train_human_indices_pg, idx_to_node, num_edges_to_add=50):
    poison_graph = original_graph.copy()
    train_bots_nx = map_indices_to_nodes(train_bot_indices_pg, idx_to_node)
    train_humans_nx = map_indices_to_nodes(train_human_indices_pg, idx_to_node)
    links_added = 0
    for _ in range(num_edges_to_add):
        if not train_bots_nx or not train_humans_nx:
            break
        bot = random.choice(train_bots_nx)
        human = random.choice(train_humans_nx)

```

d. Structural Evasion Attack

- ✓ Modify graph by adding 36 edges from 20 bots to human nodes to hide bots within the network.
- ✓ Recompute graph metrics and normalize.
- ✓ Test baseline models on this attacked graph without retraining to evaluate robustness.

e. Graph Poisoning Attack

- ✓ Add 49 edges between bots and humans in training data to corrupt the training graph.
- ✓ Recompute features and train new models on poisoned data for 100 epochs.
- ✓ Evaluate on poisoned test and clean test data.

f. Visualization

```
# Enhanced Visualization Function
def find_best_viz_target(indices_pg, data_pg, min_degree=5):
    for idx in indices_pg:
        adj_indices = data_pg.edge_index[1][data_pg.edge_index[0] == idx]
        if adj_indices.shape[0] >= min_degree:
            return idx
    return indices_pg[0]

def visualize_attack_comparison_full(graph_baseline, data_evasion, data_poison, all_bot_nodes_nx,
                                   idx_to_node, target_node_evasion, target_node_poison,
                                   evasion_targets_pg, train_bot_indices_pg):
    fig_full, ax_full = plt.subplots(figsize=(10, 8))
    ax_full.set_title('Baseline Graph: Network Structure Visualization', fontsize=16)
    try:
        center_node = max(graph_baseline.degree(), key=lambda item: item[1])[0]
        full_subgraph = nx.ego_graph(graph_baseline, center_node, radius=2)
    except:
        full_subgraph = graph_baseline.subgraph(list(graph_baseline.nodes)[:min(graph_baseline.number_of_nodes(), 400)])
    full_pos = nx.spring_layout(full_subgraph, seed=42, iterations=50)
    full_node_colors = [BOT_COLOR if node in all_bot_nodes_nx else HUMAN_COLOR for node in full_subgraph.nodes()]
    nx.draw_networkx_nodes(full_subgraph, full_pos, node_size=50, node_color=full_node_colors, ax=ax_full, edgecolors='gray')
    nx.draw_networkx_edges(full_subgraph, full_pos, width=0.5, alpha=0.3, edge_color='gray', ax=ax_full)
    full_legend_elements = [
        Patch(facecolor=BOT_COLOR, Label=f'Bots ({sum(1 for n in full_subgraph.nodes() if n in all_bot_nodes_nx)}')'),
        Patch(facecolor=HUMAN_COLOR, Label=f'Humans ({sum(1 for n in full_subgraph.nodes() if n not in all_bot_nodes_nx)}')')
    ]
    ax_full.legend(handles=full_legend_elements, loc='upper right', title="Legend")
    ax_full.axis('off')
    plt.tight_layout()
    plt.savefig(os.path.join(VIS_FOLDER, '01_baseline_full_graph.png'))
    plt.close(fig_full)
    print(f"✅ Baseline full graph visualization saved to: {os.path.join(VIS_FOLDER, '01_baseline_full_graph.png')}")
```

```
def visualize_attack_comparison_full(graph_baseline, data_evasion, data_poison, all_bot_nodes_nx,
                                   fig, axes = plt.subplots(1, 3, figsize=(18, 6))):
    fig.suptitle('Ego-Network Visualization of Bot Nodes Before and After Attacks', fontsize=16)
    graph_evasion = get_nx_graph_from_pyg_data(data_evasion, idx_to_node)
    graph_poison = get_nx_graph_from_pyg_data(data_poison, idx_to_node)
    evasion_targets_nx = map_indices_to_nodes(evasion_targets_pg, idx_to_node)
    poison_targets_nx = map_indices_to_nodes(train_bot_indices_pg, idx_to_node)
    # Plot 1: Baseline (Original)
    visualize_egonet_single(
        graph_baseline, target_node_evasion, axes[0],
        f'1. Baseline Structure ', all_bot_nodes_nx,
        highlight_color=TARGET_COLOR,
        attack_highlight_color=ATTACK_HIGHLIGHT_COLOR
    )
    # Plot 2: Structural Evasion
    visualize_egonet_single(
        graph_evasion, target_node_evasion, axes[1],
        '2. Structural Evasion ', all_bot_nodes_nx,
        highlight_color=TARGET_COLOR,
        attacked_nodes=evasion_targets_nx,
        attack_highlight_color=ATTACK_HIGHLIGHT_COLOR
    )
    # Plot 3: Graph Poisoning
    visualize_egonet_single(
        graph_poison, target_node_poison, axes[2],
        f'3. Graph Poisoning ', all_bot_nodes_nx,
        highlight_color=TARGET_COLOR,
        attacked_nodes=poison_targets_nx,
        attack_highlight_color=ATTACK_HIGHLIGHT_COLOR
    )
    legend_elements = [
        Patch(facecolor=BOT_COLOR, Label='Bot Node'),
        Patch(facecolor=HUMAN_COLOR, Label='Human Node'),
```

- Full graph visualizations with node colors by label.
- Ego-network comparisons before and after attacks to show structural changes around bots.

Graph Metrics Comparison (Baseline vs Attacks)

Graph Metrics:

- Degree: number of connections (bots often have weird degree patterns).
- Clustering Coefficient: how tightly a node's neighbors connect.
- Betweenness Centrality: how much a node sits between groups.
- Eigenvector Centrality: how "important" a node is in the network.
- Community Label: which group the node belongs to.

These help the model learn structural differences between bots and humans.

Metric	Baseline Mean	Evasion Mean	Poisoning Mean
Degree	43.691	43.709	43.715
Clustering Coefficient	0.606	0.605	0.604
Betweenness Centrality	0.000668	0.000665	0.000655
Eigenvector Centrality	0.003891	0.003892	0.003895
Communities (mean ID)	5.74	6.92	5.74

- Metrics mostly stable, except community structure shifted slightly after evasion attack (higher community count).
- Minor changes indicate attacks are subtle but still effective.

Model Performance Comparison

Accuracy

How many nodes were classified correctly overall.

Recall (Bots/Humans)

How many real bots/humans the model actually found.

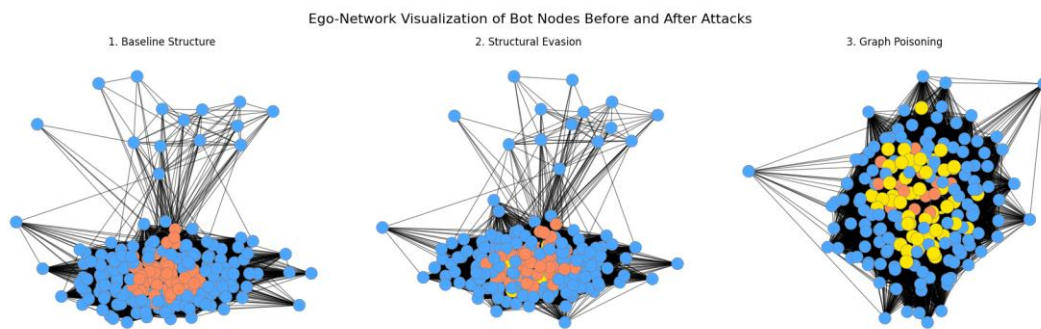
F1-Score (Bots/Humans)

How balanced the model is between catching bots and avoiding mistakes.

Model	Scenario	Accuracy	F1 (Bot)	Recall (Bot)	F1 (Human)	Recall (Human)
GraphSAGE	Baseline (clean test)	0.962	0.721	1.000	0.979	0.960
GraphSAGE	Evasion Attack (attacked test)	0.947	0.583	0.750	0.972	0.957
GraphSAGE	Poisoning Attack (poisoned test)	0.948	0.650	0.975	0.972	0.947
GraphSAGE	Poisoning Attack (clean test)	0.934	0.583	0.925	0.964	0.935
GCN	Baseline (clean test)	0.885	0.415	0.825	0.936	0.888
GCN	Evasion Attack (attacked test)	0.904	0.371	0.575	0.948	0.921
GCN	Poisoning Attack (poisoned test)	0.886	0.418	0.825	0.937	0.889
GCN	Poisoning Attack (clean test)	0.826	0.319	0.825	0.900	0.826

- Baseline models perform best on clean data.
- Structural evasion attack significantly drops bot F1 and recall, esp for GCN.
- Poisoning attack also reduces bot detection performance ,because it corrupts the training graph, so the model learns wrong patterns.
- GraphSAGE outperforms GCN overall, showing better robustness.

Explanation of Your Graph Comparison



Baseline Structure

- Bots are clustered together (orange).
- Humans are more spread out (blue).
- Clear separation:
 - Bots have low + high degrees.
 - Humans form dense communities.

Structural Evasion

- You can see new yellow nodes around bots (human neighbors).
- Bot cluster becomes less isolated.
- Bots look more integrated in the community.

Graph Poisoning

- Training graph becomes extremely dense.
- Many new links (yellow nodes mixed everywhere).
- Bots are no longer distinguishable in the structure.

Summary

- Structural evasion attack is effective in camouflaging bots by manipulating the network structure, lowering bot detection performance.
- Poisoning attack corrupts training data, degrading model ability to learn bot features.
- GraphSAGE is generally more resilient than GCN for this task.
- Visualizations illustrate how attacks subtly change network connections around bots, explaining performance drops.
- Future work: explore advanced defense techniques and more complex attacks.